

DATABASE MANAGEMENT SYSTEM

SMART CAMPUS MANAGEMENT SYSTEM

Complete Project Reference Report

Submitted for Academic Evaluation

Submitted by	Vishal Prajapati
Roll Number	FDAI044-A
Class	FY-BSC AI
Subject	Database Management Systems (DBMS)
Faculty In-charge	Onkar Raotole
Submission Date	31 August 2026
Important print instruction Write the short explanatory portions manually. Print the marked SQL-code and query-output assets from the assets folder and attach them as evidence; do not handwrite the marked screenshots.	

TABLE OF CONTENTS / INDEX

Part / Section	What the student writes or attaches
Cover Page	Student details and academic information
Part A – Requirement Analysis	Objectives, users, functional requirements, non-functional requirements
Part B – Database Design	Chosen modules, ER diagram, relational schema, keys, and 3NF
Part C – Database Implementation	Database setup, table definitions, and constraints
Part D – Data Entry	Insert strategy and record-count evidence
Part E – SQL Queries	45 queries; each query is followed by its output screenshot
Part F – Report	Challenges, learning outcomes, conclusion, and references
Printable Asset Index	Source-code screenshots and all query-output screenshots

How to use this reference

The report is intentionally detailed so the student can copy the concise written explanations. Anything labelled [PRINT THIS ASSET] is supplied in the assets folder for printing and attaching.

PART A – REQUIREMENT ANALYSIS

A1. Objectives

The objective is to design and implement a relational Smart Campus Management System. It centralizes academic and placement information and applies keys, constraints, normalization, and SQL queries for reliable data management.

A2. Users of the System

User	Responsibilities
Administrator	Maintains database structure, departments, users, and records.
Faculty	Views course rosters and records attendance and marks.
Student	Views profile, attendance, results, and placement information.
Placement Officer	Manages drives, eligibility, and application status.

A3. Functional Requirements

- The system shall store department, student, faculty, course, attendance, examination, result, and placement information.
- The system shall support insertion, retrieval, update, deletion, and schema alteration operations.
- The system shall answer analytical questions using aggregate, JOIN, nested, and pattern-matching queries.

A4. Non-Functional Requirements

Requirement	Short description
Integrity	Use PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, and DEFAULT constraints.
Performance	Support routine reports using efficient relational queries.
Scalability	Allow more students, courses, departments, and placement drives.
Maintainability	Use clear names, normalized tables, and documented SQL.

PART B – DATABASE DESIGN

B1. Selected Modules

Module	Tables	Purpose
Student Management	Student	Identity, admission, contact, and CGPA
Faculty Management	Faculty	Faculty, department, designation, and salary
Attendance	Attendance	Student attendance by course and date
Examination and Result	Examination, Result	Exam schedules, marks, and grades
Placement Cell	Placement_Drive, Placement_Record	Recruitment drives and applications

B2. Relationship Summary

Department connects to Student, Faculty, and Course. Course connects to Attendance and Examination. Result connects students with examinations. Placement_Record connects students with Placement_Drive. Foreign keys preserve these relationships.

SMART CAMPUS DATABASE WORKFLOW

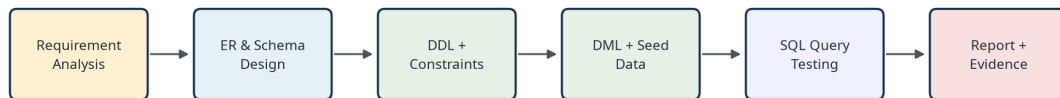


Figure B1. Database project workflow.

[PRINT THIS ASSET: [assets/smart_campus_workflow.png](#)]

SMART CAMPUS MANAGEMENT SYSTEM — ER DIAGRAM

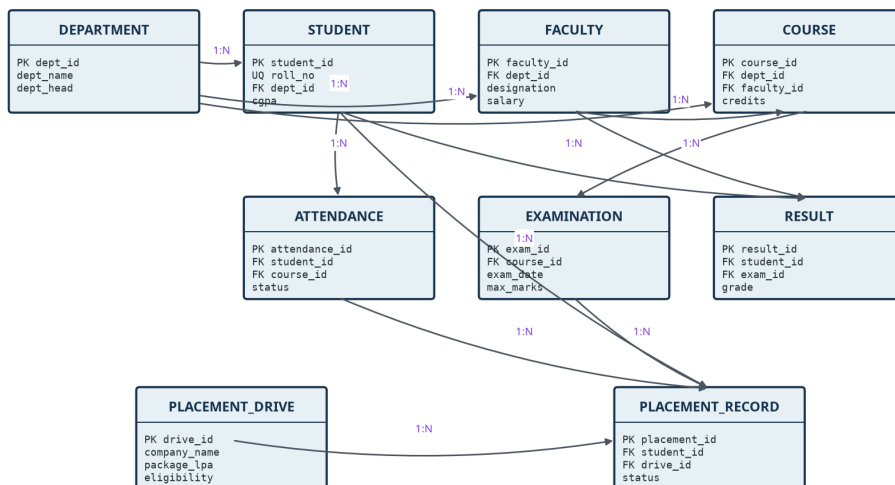


Figure B2. Entity-Relationship diagram.

[PRINT THIS ASSET: [assets/smart_campus_er_diagram.png](#)]

PART B – RELATIONAL SCHEMA, KEYS, AND NORMALIZATION

B3. Relational Schema and Key Mapping

Table	Primary key	Foreign key mapping
Department	dept_id	—
Student	student_id	dept_id → Department
Faculty	faculty_id	dept_id → Department
Course	course_id	dept_id → Department; faculty_id → Faculty
Attendance	attendance_id	student_id → Student; course_id → Course
Examination	exam_id	course_id → Course
Result	result_id	student_id → Student; exam_id → Examination
Placement_Drive	drive_id	—
Placement_Record	placement_id	student_id → Student; drive_id → Placement_Drive

B4. Normalization up to Third Normal Form

Normal form	Application
1NF	Each field contains one atomic value and every row has a primary key.
2NF	Non-key attributes depend on the complete key; transaction tables use unique combinations.
3NF	Department and faculty information are separated so non-key attributes do not depend on other non-key attributes.

Short handwritten explanation

Normalization reduces repetition and prevents update, insertion, and deletion anomalies.

PART C – DATABASE IMPLEMENTATION

C1. Database Setup

```
01 DROP DATABASE IF EXISTS smart_campus_db;
02 CREATE DATABASE smart_campus_db;
03 USE smart_campus_db;
```

C2. Representative Table Definition

```
01 CREATE TABLE Student (
02   student_id INT PRIMARY KEY AUTO_INCREMENT,
03   roll_no VARCHAR(15) NOT NULL UNIQUE,
04   full_name VARCHAR(60) NOT NULL,
05   dept_id INT,
06   cgpa DECIMAL(3,2) DEFAULT 0.00 CHECK (cgpa BETWEEN 0 AND 10),
07   FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
08     ON DELETE SET NULL
09 );
```

C3. Constraint Checklist

Constraint	Example	Purpose
PRIMARY KEY	student_id	Unique row identity
FOREIGN KEY	REFERENCES Department(dept_id)	Referential integrity
UNIQUE	roll_no UNIQUE	Prevents duplicate values
NOT NULL	full_name NOT NULL	Requires essential data
CHECK	cgpa BETWEEN 0 AND 10	Validates allowed range
DEFAULT	status DEFAULT 'Applied'	Safe fallback value

The complete DDL is available in the attached SQL source file and in the printable source-code screenshots listed later in this report.

PART D – DATA ENTRY

Seed data is inserted after the parent tables are created. This allows all foreign-key references to resolve correctly. The initial insert blocks in the source file provide the following record coverage:

Table	Initial inserted records	Use
Department	6	Department master data
Student	20	Student records
Faculty	20	Faculty records
Course	20	Course offerings
Attendance	30	Attendance events
Examination	20	Exam schedules
Result	25	Marks and grades
Placement_Drive	15	Recruitment drives
Placement_Record	20	Applications

D1. Data Entry Logic

1. Insert department records before student, faculty, and course records.
2. Insert student and faculty records with unique roll numbers and emails.
3. Insert courses and then add attendance, examination, result, and placement records.
4. Use valid foreign-key values and allowed status, gender, CGPA, and salary ranges.

Evidence note

The UPDATE and DELETE demonstrations are separate operations. Therefore, the final database snapshot after executing the full script may have fewer rows in tables affected by demonstration DELETE statements.

PART E – SQL QUERIES

The teacher requires 45 SQL queries. The source file includes exactly 45 statements divided into eight categories. For every query below, the SQL statement is shown first and its independently verified output screenshot is placed immediately below it.

Category	Required	Included
Retrieval Queries	10	10
Aggregate Function Queries	5	5
JOIN Queries	5	5
Nested / Subquery Queries	5	5
UPDATE Queries	5	5
DELETE Queries	5	5
ALTER TABLE Queries	5	5
Pattern Matching / LIKE Queries	5	5

Print convention

Every query screenshot has a matching file inside assets/query_outputs/. Print the file named in the bracket immediately below the screenshot and attach it to the written report.

PART E – SQL QUERY EVIDENCE

Query 1.1 – Show all students

Category: Retrieval. Purpose: Show all students. The statement below is taken from the published SQL source.

```
01 SELECT * FROM Student;
```

smart_campus_db — Query 1.1 output

Write-time printable evidence | Retrieval | page 1/1

```
-- Query 1.1 | Retrieval
Show all students
$ mysql --table -D smart_campus_db
SQL
SELECT * FROM Student;
OUTPUT:
```

student_id	roll_no	full_name	dept_id	admission_year	email	phone	dob	gender	cgpa
1	FA01001	Krishna Sharma	3	2023	krishna.sharma@campus.edu.in	9130678513	2004-12-04	M	8.24
2	FA01002	Aditya Sharma	1	2023	aditya.sharma@campus.edu.in	9149611734	2003-09-07	F	6.81
3	FA01003	Meera Reddy	1	2023	meera.reddy@campus.edu.in	9649622470	2006-06-09	M	6.82
4	FA01004	Karan Shah	1	2024	karan.shah@campus.edu.in	9699484023	2005-06-28	F	9.87
5	FA01005	Tamiz Malhotra	1	2024	tamiz.malhotra@campus.edu.in	9104611046	2005-11-29	F	8.19
6	FA01006	Arjun Verma	2	2024	arjun.verma@campus.edu.in	9185673588	2004-02-13	F	7.77
7	FA01007	Aditi Gupta	3	2024	aditi.gupta@campus.edu.in	9229595955	2005-12-22	M	8.31
8	FA01008	Ananya Malhotra	2	2023	ananya.malhotra@campus.edu.in	9595343124	2005-05-21	M	8.60
9	FA01009	Vidhan Nair	1	2024	vidhan.nair@campus.edu.in	9539347414	2005-02-07	F	6.81
10	FA01010	Dev Kulkarni	5	2023	dev.kulkarni@campus.edu.in	9104412915	2004-04-24	F	8.84
11	FA01011	Neha Bhat	5	2024	neha.bhat@campus.edu.in	9155683076	2005-09-16	M	8.87
12	FA01012	Krishna Nohia	2	2025	krishna.nohia@campus.edu.in	9536290818	2003-07-13	F	8.91
13	FA01013	Sirvan Sharma	1	2025	sirvan.sharma@campus.edu.in	9675567503	2005-11-11	M	7.12
14	FA01014	Ananya Pillai	1	2025	ananya.pillai@campus.edu.in	9672531214	2005-09-25	M	7.93
15	FA01015	Reyansh Rao	2	2023	reyansh.rao@campus.edu.in	9501480535	2004-09-25	M	8.28
16	FA01016	Dev Sharma	1	2024	dev.sharma@campus.edu.in	9602948867	2005-04-02	M	9.34
17	FA01017	Sai Patel	3	2023	sai.patel@campus.edu.in	9610896053	2005-02-22	F	9.90
18	FA01018	Ananya Reddy	5	2023	ananya.reddy@campus.edu.in	9679664766	2004-12-10	F	9.78
19	FA01019	Aditi Pillai	5	2023	aditi.pillai@campus.edu.in	9066388633	2004-02-11	M	8.24
20	FA01020	Kiara Shutt	2	2023	kiara.shutt@campus.edu.in	9176228245	2003-04-03	M	9.27

Output 1.1.1. Verified MySQL output for Query 1.1.

[PRINT THIS ASSET: assets/query_outputs/q01_01_retrieval_output_01.png]

Query 1.2 – Show name, roll number and cgpa of students, highest cgpa first

Category: Retrieval. Purpose: Show name, roll number and cgpa of students, highest cgpa first. The statement below is taken from the published SQL source.

```
01 SELECT full_name, roll_no, cgpa FROM Student ORDER BY cgpa DESC;
```

smart_campus_db — Query 1.2 output

Write-time printable evidence | Retrieval | page 1/1

```
-- Query 1.2 | Retrieval
Show name, roll number and cgpa of students, highest cgpa first
$ mysql --table -D smart_campus_db
SQL
SELECT full_name, roll_no, cgpa FROM Student ORDER BY cgpa DESC;
OUTPUT:
```

full_name	roll_no	cgpa
Ananya Reddy	FA01018	9.78
Sai Patel	FA01017	9.90
Dev Sharma	FA01016	9.34
Kiara Shutt	FA01020	9.27
Karan Shah	FA01004	9.87
Neha Bhat	FA01011	8.87
Dev Kulkarni	FA01010	8.84
Ananya Malhotra	FA01008	8.60
Aditi Gupta	FA01007	8.31
Reyansh Rao	FA01015	8.28
Aditi Pillai	FA01019	8.24
Krishna Sharma	FA01001	8.24
Tamiz Malhotra	FA01005	8.19
Krishna Nohia	FA01012	8.91
Ananya Pillai	FA01014	7.93
Arjun Verma	FA01006	7.77
Sirvan Sharma	FA01013	7.12
Aditya Sharma	FA01002	6.84
Meera Reddy	FA01003	6.82
Vidhan Nair	FA01009	6.81

Output 1.2.1. Verified MySQL output for Query 1.2.

[PRINT THIS ASSET: assets/query_outputs/q01_02_retrieval_output_01.png]

PART E – SQL QUERY EVIDENCE

Query 1.3 – Show all faculty members from the Artificial Intelligence department

Category: Retrieval. Purpose: Show all faculty members from the Artificial Intelligence department. The statement below is taken from the published SQL source.

```
01 SELECT full_name, designation, email
02 FROM Faculty
03 WHERE dept_id = (SELECT dept_id FROM Department WHERE dept_name = 'Artificial Intelligence');
```

smart_campus_db – Query 1.3 output

```
Write-time printable evidence | Retrieval | page 1/1

-- Query 1.3 | Retrieval
Show all faculty members from the Artificial Intelligence department
$ mysql --table -D smart_campus_db
SQL
SELECT full_name, designation, email
FROM Faculty
WHERE dept_id = (SELECT dept_id FROM Department WHERE dept_name = 'Artificial Intelligence');
OUTPUT
+-----+-----+-----+
| full_name | designation | email |
+-----+-----+-----+
| Prof. Arjun Kapoor | Professor | arjun.kapoor@campus.edu.in |
| Prof. Nitiran Agarwal | Associate Professor | nitiran.agarwal@campus.edu.in |
| Prof. Nikhil Patel | Professor | nikhil.patel@campus.edu.in |
| Prof. Parit Singh | Professor | parit.singh@campus.edu.in |
+-----+-----+-----+
```

Output 1.3.1. Verified MySQL output for Query 1.3.

[PRINT THIS ASSET: assets/query_outputs/q01_03_retrieval_output_01.png]

Query 1.4 – Show students who took admission in 2024

Category: Retrieval. Purpose: Show students who took admission in 2024. The statement below is taken from the published SQL source.

```
01 SELECT roll_no, full_name, admission_year FROM Student WHERE admission_year = 2024;
```

smart_campus_db – Query 1.4 output

```
Write-time printable evidence | Retrieval | page 1/1

-- Query 1.4 | Retrieval
Show students who took admission in 2024
$ mysql --table -D smart_campus_db
SQL
SELECT roll_no, full_name, admission_year FROM Student WHERE admission_year = 2024;
OUTPUT
+-----+-----+-----+
| roll_no | full_name | admission_year |
+-----+-----+-----+
| FBA2004 | Karan Shah | 2024 |
| FBA2005 | Tanvi Malhotra | 2024 |
| FBA2006 | Arjun Verma | 2024 |
| FBA2007 | Aditi Gupta | 2024 |
| FBA2009 | Vidhan Nair | 2024 |
| FBA2011 | Rhea Bhatt | 2024 |
| FBA2016 | Dev Sharma | 2024 |
+-----+-----+-----+
```

Output 1.4.1. Verified MySQL output for Query 1.4.

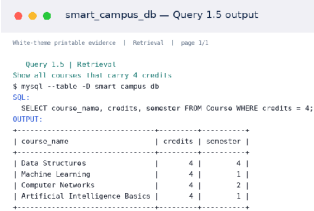
[PRINT THIS ASSET: assets/query_outputs/q01_04_retrieval_output_01.png]

PART E – SQL QUERY EVIDENCE

Query 1.5 – Show all courses that carry 4 credits

Category: Retrieval. Purpose: Show all courses that carry 4 credits. The statement below is taken from the published SQL source.

```
01 SELECT course_name, credits, semester FROM Course WHERE credits = 4;
```



```
smart_campus_db — Query 1.5 output
Write-time printable evidence | Retrieval | page 1/1

-- Query 1.5 | Retrieval
Show all courses that carry 4 credits
$ mysql --table -D smart_campus_db
SQL
SELECT course_name, credits, semester FROM Course WHERE credits = 4;
OUTPUT:
+-----+
| course_name | credits | semester |
+-----+
| Data Structures | 4 | 4 |
| Machine Learning | 4 | 1 |
| Computer Networks | 4 | 2 |
| Artificial Intelligence Basics | 4 | 1 |
+-----+
```

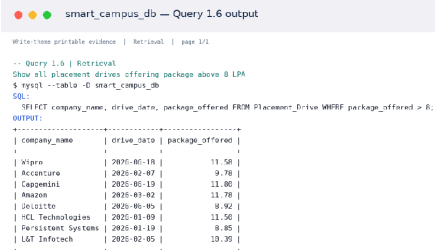
Output 1.5.1. Verified MySQL output for Query 1.5.

[PRINT THIS ASSET: assets/query_outputs/q01_05_retrieval_output_01.png]

Query 1.6 – Show all placement drives offering package above 8 LPA

Category: Retrieval. Purpose: Show all placement drives offering package above 8 LPA. The statement below is taken from the published SQL source.

```
01 SELECT company_name, drive_date, package_offered FROM Placement_Drive WHERE package_offered > 8;
```



```
smart_campus_db — Query 1.6 output
Write-time printable evidence | Retrieval | page 1/1

-- Query 1.6 | Retrieval
Show all placement drives offering package above 8 LPA
$ mysql --table -D smart_campus_db
SQL
SELECT company_name, drive_date, package_offered FROM Placement_Drive WHERE package_offered > 8;
OUTPUT:
+-----+
| company_name | drive_date | package_offered |
+-----+
| Xipre | 2025-05-18 | 11.58 |
| Accucore | 2025-02-07 | 9.78 |
| Capgemini | 2025-05-19 | 11.00 |
| Amazon | 2025-03-02 | 11.78 |
| Deloitte | 2025-06-05 | 8.92 |
| M&L Technologies | 2025-03-09 | 11.56 |
| Persistent Systems | 2025-01-19 | 8.85 |
| L&T Infotech | 2025-02-05 | 10.39 |
+-----+
```

Output 1.6.1. Verified MySQL output for Query 1.6.

[PRINT THIS ASSET: assets/query_outputs/q01_06_retrieval_output_01.png]

PART E – SQL QUERY EVIDENCE

Query 1.7 — Show top 5 students by cgpa

Category: Retrieval. Purpose: Show top 5 students by cgpa. The statement below is taken from the published SQL source.

```
01 SELECT full_name, roll_no, cgpa FROM Student ORDER BY cgpa DESC LIMIT 5;
```

smart_campus_db — Query 1.7 output

```
Write-time printable evidence | Retrieval | page 1/1

-- Query 1.7 | Retrieval
Show top 5 students by cgpa
$mysql --table -D smart_campus_db
SQL
SELECT full_name, roll_no, cgpa FROM Student ORDER BY cgpa DESC LIMIT 5;
OUTPUT:
-----
| full_name | roll_no | cgpa |
|-----|-----|-----|
| Ananya Reddy | FBA2010 | 9.78 |
| Sai Patel | FBA2011 | 9.69 |
| Dev Sharma | FBA2010 | 9.34 |
| Kiara Shett | FBA2000 | 9.27 |
| Karan Shah | FBA1004 | 9.07 |
-----
```

Output 1.7.1. Verified MySQL output for Query 1.7.

[PRINT THIS ASSET: assets/query_outputs/q01_07_retrieval_output_01.png]

Query 1.8 — Show all exams scheduled after 1st May 2026

Category: Retrieval. Purpose: Show all exams scheduled after 1st May 2026. The statement below is taken from the published SQL source.

```
01 SELECT exam_name, exam_date, max_marks FROM Examination WHERE exam_date > '2026-05-01';
```

smart_campus_db — Query 1.8 output

```
Write-time printable evidence | Retrieval | page 1/1

-- Query 1.8 | Retrieval
Show all exams scheduled after 1st May 2026
$mysql --table -D smart_campus_db
SQL
SELECT exam_name, exam_date, max_marks FROM Examination WHERE exam_date > '2026-05-01';
OUTPUT:
-----
| exam_name | exam_date | max_marks |
|-----|-----|-----|
| Unit Test 1 | 2026-07-11 | 100 |
| End Semester Exam | 2026-05-27 | 100 |
| End Semester Exam | 2026-06-18 | 100 |
| End Semester Exam | 2026-05-10 | 50 |
| Mid Semester Exam | 2026-07-01 | 100 |
| Mid Semester Exam | 2026-05-27 | 100 |
| Unit Test 1 | 2026-05-20 | 100 |
| Unit Test 2 | 2026-06-18 | 100 |
| Unit Test 2 | 2026-06-11 | 100 |
| Mid Semester Exam | 2026-06-27 | 50 |
| Unit Test 1 | 2026-05-08 | 50 |
| End Semester Exam | 2026-05-15 | 100 |
-----
```

Output 1.8.1. Verified MySQL output for Query 1.8.

[PRINT THIS ASSET: assets/query_outputs/q01_08_retrieval_output_01.png]

PART E – SQL QUERY EVIDENCE

Query 1.9 – Show student details along with their department name

Category: Retrieval. Purpose: Show student details along with their department name. The statement below is taken from the published SQL source.

```
01 SELECT s.roll_no, s.full_name, d.dept_name
02 FROM Student s, Department d
03 WHERE s.dept_id = d.dept_id;
```

smart_campus_db – Query 1.9 output

```
Write-time portable evidence | Retrieval | page 1/1

-- Query 1.9 | Retrieval
Show student details along with their department name
$ mysql --table -D smart_campus_db
SQL
  SELECT s.roll_no, s.full_name, d.dept_name
  FROM Student s, Department d
  WHERE s.dept_id = d.dept_id;
OUTPUT:
+-----+-----+-----+
| roll_no | full_name | dept_name |
+-----+-----+-----+
| FBAT000 | Arjun Varma | Artificial Intelligence |
| FBAT000 | Ananya Mahendra | Artificial Intelligence |
| FBAT012 | Krishna Mehta | Artificial Intelligence |
| FBAT015 | Raghav Rao | Artificial Intelligence |
| FBAT020 | Kaira Bhutt | Artificial Intelligence |
| FBAT010 | Dev Kulkarni | Commerce |
| FBAT011 | Neha Shett | Commerce |
| FBAT017 | Sai Patel | Commerce |
| FBAT018 | Ananya Reddy | Commerce |
| FBAT019 | Aditi Pillai | Commerce |
| FBAT003 | Aditya Sharma | Computer Science |
| FBAT003 | Meera Rooddy | Computer Science |
| FBAT004 | Karen Shah | Computer Science |
| FBAT005 | Tanvi Mahotra | Computer Science |
| FBAT005 | Vinay Nair | Computer Science |
| FBAT013 | Sivan Sharma | Computer Science |
| FBAT014 | Ananya Pillai | Computer Science |
| FBAT016 | Dev Sharma | Computer Science |
| FBAT001 | Krishna Sharma | Electronics |
| FBAT007 | Aditi Gupta | Electronics |
+-----+-----+-----+
```

Output 1.9.1. Verified MySQL output for Query 1.9.

[PRINT THIS ASSET: [assets/query_outputs/q01_09_retrieval_output_01.png](#)]

Query 1.10 – Show distinct designations used by faculty members

Category: Retrieval. Purpose: Show distinct designations used by faculty members. The statement below is taken from the published SQL source.

```
01 SELECT DISTINCT designation FROM Faculty;
02
03
04 -- *****
05 -- 2. AGGREGATE FUNCTION QUERIES (5)
06 -- *****
```

smart_campus_db – Query 1.10 output

```
Write-time portable evidence | Retrieval | page 1/1

-- Query 1.10 | Retrieval
Show distinct designations used by faculty members
$ mysql --table -D smart_campus_db
SQL
  SELECT DISTINCT designation FROM Faculty;
OUTPUT:
+-----+
| designation |
+-----+
| Professor |
| Associate Professor |
| Lecturer |
| Assistant Professor |
+-----+
```

Output 1.10.1. Verified MySQL output for Query 1.10.

[PRINT THIS ASSET: [assets/query_outputs/q01_10_retrieval_output_01.png](#)]

PART E – SQL QUERY EVIDENCE

Query 2.1 – Total number of students in the campus

Category: Aggregate Function. Purpose: Total number of students in the campus. The statement below is taken from the published SQL source.

```
01 SELECT COUNT(*) AS total_students FROM Student;
```

smart_campus_db — Query 2.1 output

```
Write-time printable evidence | Aggregate Function | page 1/1

Query 2.1 | Aggregate Function
Total number of students in the campus
$ mysql --table -D smart_campus_db
SQL
SELECT COUNT(*) AS total_students FROM Student;
OUTPUT:
+-----+
| total_students |
+-----+
| 28 |
+-----+
```

Output 2.1.1. Verified MySQL output for Query 2.1.

[PRINT THIS ASSET: [assets/query_outputs/q02_01_aggregate_output_01.png](#)]

Query 2.2 – Average cgpa of all students

Category: Aggregate Function. Purpose: Average cgpa of all students. The statement below is taken from the published SQL source.

```
01 SELECT ROUND(AVG(cgpa),2) AS average_cgpa FROM Student;
```

smart_campus_db — Query 2.2 output

```
Write-time printable evidence | Aggregate Function | page 1/1

Query 2.2 | Aggregate Function
Average cgpa of all students
$ mysql --table -D smart_campus_db
SQL
SELECT ROUND(AVG(cgpa),2) AS average_cgpa FROM Student;
OUTPUT:
+-----+
| average_cgpa |
+-----+
| 8.29 |
+-----+
```

Output 2.2.1. Verified MySQL output for Query 2.2.

[PRINT THIS ASSET: [assets/query_outputs/q02_02_aggregate_output_01.png](#)]

Query 2.3 – Highest and lowest package offered across all placement drives

Category: Aggregate Function. Purpose: Highest and lowest package offered across all placement drives. The statement below is taken from the published SQL source.

```
01 SELECT MAX(package_offered) AS highest_package, MIN(package_offered) AS lowest_package
02 FROM Placement_Drive;
```

smart_campus_db — Query 2.3 output

```
Write-time printable evidence | Aggregate Function | page 1/1

-- Query 2.3 | Aggregate Function
Highest and lowest package offered across all placement drives
$ mysql --table -D smart_campus_db
SQL
SELECT MAX(package_offered) AS highest_package, MIN(package_offered) AS lowest_package
FROM Placement_Drive;
OUTPUT:
+-----+ +-----+
| highest_package | lowest_package |
+-----+ +-----+
| 11.69 | 3.60 |
+-----+ +-----+
```

Output 2.3.1. Verified MySQL output for Query 2.3.

[PRINT THIS ASSET: [assets/query_outputs/q02_03_aggregate_output_01.png](#)]

PART E – SQL QUERY EVIDENCE

Query 2.4 – Number of students in each department

Category: Aggregate Function. Purpose: Number of students in each department. The statement below is taken from the published SQL source.

```
01 SELECT d.dept_name, COUNT(s.student_id) AS total_students
02 FROM Department d
03 LEFT JOIN Student s ON d.dept_id = s.dept_id
04 GROUP BY d.dept_name;
```

smart_campus_db — Query 2.4 output

Write-time portable evidence | Aggregate Function | page 1/1

```
Query 2.4 | Aggregate Function
Number of students in each department
$mysql --table -D smart_campus_db
SQL:
SELECT d.dept_name, COUNT(s.student_id) AS total_students
FROM Department d
LEFT JOIN Student s ON d.dept_id = s.dept_id
GROUP BY d.dept_name;
OUTPUT:
+-----+
| dept_name | total_students |
+-----+
| Artificial Intelligence | 8 |
| Commerce | 5 |
| Computer Science | 8 |
| Electronics | 2 |
| Mathematics | 0 |
| Mechanical | 0 |
+-----+
```

Output 2.4.1. Verified MySQL output for Query 2.4.

[PRINT THIS ASSET: [assets/query_outputs/q02_04_aggregate_output_01.png](#)]

Query 2.5 – Total salary paid by the campus to faculty of each department

Category: Aggregate Function. Purpose: Total salary paid by the campus to faculty of each department. The statement below is taken from the published SQL source.

```
01 SELECT d.dept_name, SUM(f.salary) AS total_salary
02 FROM Department d
03 JOIN Faculty f ON d.dept_id = f.dept_id
04 GROUP BY d.dept_name;
05
06
07 -- *****
08 -- 3. JOIN QUERIES (5)
09 -- *****
```

smart_campus_db — Query 2.5 output

Write-time portable evidence | Aggregate Function | page 1/1

```
-- Query 2.5 | Aggregate Function
Total salary paid by the campus to faculty of each department
$mysql --table -D smart_campus_db
SQL:
SELECT d.dept_name, SUM(f.salary) AS total_salary
FROM Department d
JOIN Faculty f ON d.dept_id = f.dept_id
GROUP BY d.dept_name;
*****
-- 3. JOIN QUERIES (5)
*****
OUTPUT:
+-----+
| dept_name | total_salary |
+-----+
| Artificial Intelligence | 20142.00 |
| Commerce | 160124.00 |
| Computer Science | 295553.00 |
| Electronics | 165261.00 |
| Mathematics | 212555.00 |
| Mechanical | 389953.00 |
+-----+
```

Output 2.5.1. Verified MySQL output for Query 2.5.

[PRINT THIS ASSET: [assets/query_outputs/q02_05_aggregate_output_01.png](#)]

PART E – SQL QUERY EVIDENCE

Query 3.1 – List every student with the course they attend and the class faculty (INNER JOIN, 3 tables)

Category: JOIN. Purpose: List every student with the course they attend and the class faculty (INNER JOIN, 3 tables). The statement below is taken from the published SQL source.

```
01 SELECT s.full_name AS student, c.course_name, f.full_name AS faculty
02 FROM Attendance a
03 JOIN Student s ON a.student_id = s.student_id
04 JOIN Course c ON a.course_id = c.course_id
05 JOIN Faculty f ON c.faculty_id = f.faculty_id;
```

smart_campus_db — Query 3.1 output

Write-time printable evidence | JOIN | page 1/2

```
-- Query 3.1 | JOIN
-- List every student with the course they attend and the class faculty (INNER JOIN, 3 tables)
$ mysql --table -D smart_campus_db
SQL>
mysql> select s.full_name AS student, c.course_name, f.full_name AS faculty
FROM Attendance a
JOIN Student s ON a.student_id = s.student_id
JOIN Course c ON a.course_id = c.course_id
JOIN Faculty f ON c.faculty_id = f.faculty_id;
OUTPUT
```

student	course_name	faculty
Krishna Sharma	Data Structures	Prof. Remyash Mehta
Krishna Sharma	Maths for engineering	Prof. Saamvi Malhotra
Krishna Sharma	Statistics for AI	Prof. Myra Gupta
Meera Reddy	Mobile App Development	Prof. Priya Sharma
Narain Shah	Mobile App Development	Prof. Priya Sharma
Arijun Verma	Python Programming	Prof. Arjun Kapoor
Aditi Gupta	Cloud Computing	Prof. Dr. Gupta
Ananya Malhotra	Digital Electronics	Prof. Saamvi Malhotra
Ananya Malhotra	Cyber Security	Prof. Priya Sharma
Vishvan Mair	Operating Systems	Prof. Saamvi Malhotra
Vishvan Mair	Statistics for AI	Prof. Myra Gupta
Dev Kulkarni	Statistics for AI	Prof. Myra Gupta
Neha Bharti	Software Engineering	Prof. Sneha Mehta
Neha Bharti	Computer Graphics	Prof. Dr. Gupta
Saran Sharma	Operating Systems	Prof. Saamvi Malhotra
Saran Sharma	Digital Electronics	Prof. Saamvi Malhotra
Saran Sharma	Java Programming	Prof. Raj Verma
Ananya Pillai	Linear Algebra	Prof. Remyash Chavan
Ananya Pillai	Computer Graphics	Prof. Dr. Gupta
Remyash Rao	Python Programming	Prof. Arjun Kapoor
Remyash Rao	Web Development	Prof. Kanya Shett
Remyash Rao	Computer Graphics	Prof. Dr. Gupta
Dev Sharma	Mobile App Development	Prof. Priya Sharma
Sai Patel	Mobile App Development	Prof. Priya Sharma
Sai Patel	Linear Algebra	Prof. Remyash Chavan
Ananya Reddy	Database Management Systems	Prof. Sieran Agarwal
Aditi Pillai	Web Development	Prof. Kanya Shett
Aditi Pillai	Computer Graphics	Prof. Dr. Gupta
Klara Shett	Software Engineering	Prof. Sneha Mehta
Klara Shett	Data Analytics	Prof. Remyash Verma

Output 3.1.1. Verified MySQL output for Query 3.1.

[PRINT THIS ASSET: assets/query_outputs/q03_01_join_output_01.png]

smart_campus_db — Query 3.1 output

Write-time printable evidence | JOIN | page 2/2

Output 3.1.2. Verified MySQL output for Query 3.1.

[PRINT THIS ASSET: assets/query_outputs/q03_01_join_output_02.png]

PART E – SQL QUERY EVIDENCE

Query 3.2 – List each student's marks together with the exam and course name

Category: JOIN. Purpose: List each student's marks together with the exam and course name. The statement below is taken from the published SQL source.

```
01 SELECT s.full_name AS student, e.exam_name, c.course_name, r.marks_obtained, r.grade
02 FROM Result r
03 JOIN Student s ON r.student_id = s.student_id
04 JOIN Examination e ON r.exam_id = e.exam_id
05 JOIN Course c ON e.course_id = c.course_id;
```

smart_campus_db — Query 3.2 output

Write-time portable evidence | JOIN | page 1/1

```
-- Query 3.2 | JOIN
List each student's marks together with the exam and course name
$ mysql --table -D smart_campus_db
SQL
SELECT s.full_name AS student, e.exam_name, c.course_name, r.marks_obtained, r.grade
FROM Result r
JOIN Student s ON r.student_id = s.student_id
JOIN Examination e ON r.exam_id = e.exam_id
JOIN Course c ON e.course_id = c.course_id;
OUTPUT
+-----+-----+-----+-----+-----+
| student | exam_name | course_name | marks_obtained | grade |
+-----+-----+-----+-----+-----+
| Neha Bharti | Unit Test 1 | Data Analytics | 91.00 | A+ |
| Krishna Mohita | End Semester Exam | Object Oriented Programming | 70.40 | D |
| Ananya Malhotra | End Semester Exam | Object Oriented Programming | 56.70 | D |
| Aditi Gupta | End Semester Exam | Object Oriented Programming | 54.50 | D |
| Dev Sharma | End Semester Exam | Database Management System | 66.00 | C |
| Ananya Pillai | Mid Semester Exam | Data Analytics | 84.00 | A |
| Nayansh Rao | Unit Test 2 | Computer Graphics | 42.50 | A |
| Aditi Pillai | Unit Test 1 | Data Analytics | 50.30 | D |
| Neha Bharti | Unit Test 1 | Data Analytics | 92.00 | A+ |
| Meera Nundy | Unit Test 2 | Data Structures | 53.30 | D |
| Ananya Malhotra | Unit Test 2 | Data Structures | 70.00 | B |
| Dev Sharma | Unit Test 2 | Mobile App Development | 50.00 | D |
| Ananya Reddy | Unit Test 2 | Mobile App Development | 61.10 | C |
| Aditi Gupta | Mid Semester Exam | Data Analytics | 61.00 | A |
| Krishna Mohita | Unit Test 1 | Data Structures | 40.00 | A+ |
| Krishna Mohita | Unit Test 1 | Computer Graphics | 41.20 | A |
| Dev Sharma | Unit Test 1 | Computer Graphics | 43.00 | A |
| Ananya Reddy | End Semester Exam | Machine Learning | 67.00 | C |
| Ananya Pillai | End Semester Exam | Machine Learning | 73.00 | B |
| Ananya Malhotra | End Semester Exam | Machine Learning | 70.00 | D |
| Krishna Mohita | End Semester Exam | Machine Learning | 73.20 | B |
| Krishna Mohita | Mid Semester Exam | Machine Learning | 65.70 | C |
| Vibhaan Mair | Mid Semester Exam | Machine Learning | 40.00 | F |
| Vibhaan Mair | Unit Test 1 | Digital Electronics | 44.20 | A |
| Aditi Pillai | Unit Test 1 | Digital Electronics | 40.00 | A |
```

Output 3.2.1. Verified MySQL output for Query 3.2.

[PRINT THIS ASSET: [assets/query_outputs/q03_02_join_output_01.png](#)]

PART E – SQL QUERY EVIDENCE

Query 3.3 – LEFT JOIN - show every student even if they never applied for placement

Category: JOIN. Purpose: LEFT JOIN - show every student even if they never applied for placement. The statement below is taken from the published SQL source.

```
01 SELECT s.full_name, pd.company_name, pr.status
02 FROM Student s
03 LEFT JOIN Placement_Record pr ON s.student_id = pr.student_id
04 LEFT JOIN Placement_Drive pd ON pr.drive_id = pd.drive_id
05 ORDER BY s.full_name;
```

smart_campus_db — Query 3.3 output

Write-time printable evidence | JOIN | page 1/1

```
-- Query 3.3 | JOIN
LEFT JOIN - show every student even if they never applied for placement
$ mysql --table -D smart_campus_db
```

```
SQL
SELECT s.full_name, pd.company_name, pr.status
FROM Student s
LEFT JOIN Placement_Record pr ON s.student_id = pr.student_id
LEFT JOIN Placement_Drive pd ON pr.drive_id = pd.drive_id
ORDER BY s.full_name;
```

```
OUTPUT
+-----+
| full_name | company_name | status |
+-----+
| Aditi Gupta | Capgemini | Selected |
| Aditi Gupta | IBM India | Applied |
| Aditi Pillai | Tech Mahindra | Selected |
| Aditya Sharma | EY | Rejected |
| Ananya Mulhundra | Tech Mahindra | Applied |
| Ananya Pillai | Capgemini | Shortlisted |
| Ananya Pillai | EY | Shortlisted |
| Ananya Reddy | NULL | NULL |
| Arjun Verma | Accenture | Applied |
| Dev Kulkarni | TCS | Applied |
| Dev Kulkarni | HCL Technologies | Rejected |
| Dev Kulkarni | Persistent Systems | Shortlisted |
| Dev Sharma | NULL | NULL |
| Karan Shah | Zoho | Applied |
| Kiana Bhatt | Accenture | Selected |
| Krishna Rohra | NULL | NULL |
| Krishna Sharma | NULL | NULL |
| Meera Reddy | Wipro | Applied |
| Meera Reddy | Accenture | Applied |
| Meera Reddy | Tech Mahindra | Applied |
| Naha Bhatt | NULL | NULL |
| Nayanah Rao | ICL Technologies | Shortlisted |
| Sai Patel | NULL | NULL |
| Suman Sharma | Amazon | Rejected |
| Tanvi Mahotra | NULL | NULL |
| Vibhansh Nair | Zoho | Selected |
| Vibhansh Nair | Persistent Systems | Applied |
+-----+
```

Output 3.3.1. Verified MySQL output for Query 3.3.

[PRINT THIS ASSET: [assets/query_outputs/q03_03_join_output_01.png](#)]

PART E – SQL QUERY EVIDENCE

Query 3.4 – List courses along with the department and the faculty teaching them

Category: JOIN. Purpose: List courses along with the department and the faculty teaching them. The statement below is taken from the published SQL source.

```
01 SELECT c.course_name, d.dept_name, f.full_name AS faculty_name
02 FROM Course c
03 JOIN Department d ON c.dept_id = d.dept_id
04 JOIN Faculty f ON c.faculty_id = f.faculty_id;
```

smart_campus_db — Query 3.4 output

Write-time portable evidence | 2018 | page 1/1

```
-- Query 3.4 | 2018
List courses along with the department and the faculty teaching them
$ mysql --table -D smart_campus_db
SQL:
SELECT c.course_name, d.dept_name, f.full_name AS faculty_name
FROM Course c
JOIN Department d ON c.dept_id = d.dept_id
JOIN Faculty f ON c.faculty_id = f.faculty_id;
OUTPUT:
+-----+-----+-----+
| course_name | dept_name | faculty_name |
+-----+-----+-----+
| Database Management Systems | Artificial Intelligence | Prof. Sitaran Agarwal |
| Software Engineering | Database | Prof. Snehil Mehta |
| Python Programming | Commerce | Prof. Arjun Kapoor |
| Computer Graphics | Computer Science | Prof. Dhruv Gupta |
| Statistics for AI | Electronics | Prof. Priya Gupta |
| Computer Networks | Electronics | Prof. Parv Singh |
| Web Development | Electronics | Prof. Kavya Mehta |
| Discrete Mathematics | Electronics | Prof. Ravish Mehta |
| Java Programming | Electronics | Prof. Raj Verma |
| Linear Algebra | Mathematics | Prof. Ravish Chavan |
| Machine Learning | Mathematics | Prof. Saanvi Mehta |
| Artificial Intelligence Basics | Mathematics | Prof. Priya Sharma |
| Digital Electronics | Mathematics | Prof. Saanvi Mehta |
| Cloud Computing | Mathematics | Prof. Dhruv Gupta |
| Cyber Security | Mathematics | Prof. Priya Sharma |
| Data Analytics | Mathematics | Prof. Ravish Verma |
| Object Oriented Programming | Mathematics | Prof. Isha Singh |
| Data Structures | Mechanical | Prof. Ravish Mehta |
| Operating Systems | Mechanical | Prof. Saanvi Mehta |
| Mobile App Development | Mechanical | Prof. Priya Sharma |
+-----+-----+-----+
```

Output 3.4.1. Verified MySQL output for Query 3.4.

[PRINT THIS ASSET: assets/query_outputs/q03_04_join_output_01.png]

PART E – SQL QUERY EVIDENCE

Query 3.5 – RIGHT JOIN - show all placement drives and the students placed in them, if any

Category: JOIN. Purpose: RIGHT JOIN - show all placement drives and the students placed in them, if any. The statement below is taken from the published SQL source.

```
01 SELECT pd.company_name, s.full_name, pr.status
02 FROM Placement_Record pr
03 RIGHT JOIN Placement_Drive pd ON pr.drive_id = pd.drive_id
04 LEFT JOIN Student s ON pr.student_id = s.student_id;
05
06
07 -- *****
08 -- 4. NESTED (SUBQUERY) QUERIES (5)
09 -- *****
```

smart_campus_db — Query 3.5 output

Write these printable evidence | 2028 | page 3/1

```
-- Query 3.5 : 2028
RIGHT JOIN - show all placement drives and the students placed in them, if any
$ mysql --table -D smart_campus_db
SQL
SELECT pd.company_name, s.full_name, pr.status
FROM Placement_Record pr
RIGHT JOIN Placement_Drive pd ON pr.drive_id = pd.drive_id
LEFT JOIN Student s ON pr.student_id = s.student_id;
```

```
4. NESTED (SUBQUERY) QUERIES (5)
..
OUTPUT:
```

company_name	full_name	status
TCS	Dev Kulkarni	Applied
Infosys	NULL	NULL
Wipro	Meero Reddy	Applied
Accenture	Maera Reddy	Applied
Accenture	Klara Shwett	Selected
Accenture	Arjun Verma	Applied
Cognizant	Ananya Pillai	Shortlisted
Cognizant	Aditi Gupta	Selected
Cognizant	NULL	NULL
TBN India	Aditi Gupta	Applied
Amazon	Sirran Sharma	Rejected
Deloitte	NULL	NULL
BY	Ananya Pillai	Shortlisted
BY	Aditya Sharma	Rejected
Tech Mahindra	Maera Reddy	Applied
Tech Mahindra	Aditi Pillai	Selected
Tech Mahindra	Romya Nathrao	Applied
HCL Technologies	Haryash Rao	Shortlisted
HCL Technologies	Dev Kulkarni	Rejected
Zoho	Karan Swan	Applied
Zoho	Vishoon Nair	Selected
Persistent Systems	Dev Kulkarni	Shortlisted
Persistent Systems	Vishoon Nair	Applied
LAT Infotech	NULL	NULL

Output 3.5.1. Verified MySQL output for Query 3.5.

[PRINT THIS ASSET: assets/query_outputs/q03_05_join_output_01.png]

PART E – SQL QUERY EVIDENCE

Query 4.1 – Students whose cgpa is above the campus average

Category: Nested Query. Purpose: Students whose cgpa is above the campus average. The statement below is taken from the published SQL source.

```
01 SELECT full_name, cgpa
02 FROM Student
03 WHERE cgpa > (SELECT AVG(cgpa) FROM Student);
```

smart_campus_db – Query 4.1 output

```
Write-time printable evidence | Nested Query | page 1/1

-- Query 4.1 | Nested Query
Students whose cgpa is above the campus average
$ mysql --table -D smart_campus_db
SQL
SELECT full_name, cgpa
FROM Student
WHERE cgpa > (SELECT AVG(cgpa) FROM Student);
OUTPUT
+-----+
| full_name | cgpa |
+-----+
| Karan Shah | 9.87 |
| Aditi Gupta | 8.91 |
| Ananya Malhotra | 8.60 |
| Dev Kulkarni | 9.84 |
| Neha Bharti | 8.87 |
| Dev Sharma | 9.34 |
| Sai Patel | 9.65 |
| Ananya Baidya | 9.79 |
| Kiara Shetty | 9.27 |
+-----+
```

Output 4.1.1. Verified MySQL output for Query 4.1.

[PRINT THIS ASSET: [assets/query_outputs/q04_01_nested_query_output_01.png](#)]

Query 4.2 – Faculty who earn more than the average salary of their own department

Category: Nested Query. Purpose: Faculty who earn more than the average salary of their own department. The statement below is taken from the published SQL source.

```
01 SELECT full_name, dept_id, salary
02 FROM Faculty f
03 WHERE salary > (
04     SELECT AVG(salary) FROM Faculty WHERE dept_id = f.dept_id
05 );
```

smart_campus_db – Query 4.2 output

```
Write-time printable evidence | Nested Query | page 1/1

-- Query 4.2 | Nested Query
Faculty who earn more than the average salary of their own department
$ mysql --table -D smart_campus_db
SQL
SELECT full_name, dept_id, salary
FROM Faculty f
WHERE salary > (
    SELECT AVG(salary) FROM Faculty WHERE dept_id = f.dept_id
);
OUTPUT
+-----+
| full_name | dept_id | salary |
+-----+
| Prof. Arjun Kapoor | 2 | 76012.00 |
| Prof. Rajnish Verma | 4 | 101470.00 |
| Prof. Arjun Pillai | 5 | 87720.00 |
| Prof. Priya Sharma | 4 | 66454.00 |
| Prof. Priya Kulkarni | 1 | 83962.00 |
| Prof. Isha Singh | 6 | 98249.00 |
| Prof. Rajnish Mehta | 3 | 93655.00 |
| Prof. Fari Singh | 2 | 69055.00 |
+-----+
```

Output 4.2.1. Verified MySQL output for Query 4.2.

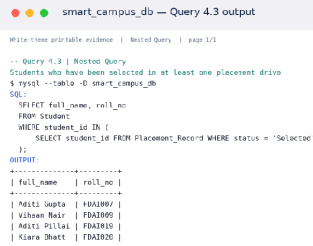
[PRINT THIS ASSET: [assets/query_outputs/q04_02_nested_query_output_01.png](#)]

PART E – SQL QUERY EVIDENCE

Query 4.3 – Students who have been selected in at least one placement drive

Category: Nested Query. Purpose: Students who have been selected in at least one placement drive. The statement below is taken from the published SQL source.

```
01 SELECT full_name, roll_no
02 FROM Student
03 WHERE student_id IN (
04     SELECT student_id FROM Placement_Record WHERE status = 'Selected'
05 );
```



smart_campus_db — Query 4.3 output

Write-time portable evidence | Nested Query | page 1/1

```
-- Query 4.3 | Nested Query
Students who have been selected in at least one placement drive
$ mysql --table -D smart_campus_db
SQL:
SELECT full_name, roll_no
FROM Student
WHERE student_id IN (
    SELECT student_id FROM Placement_Record WHERE status = 'Selected'
);
OUTPUT:
+-----+-----+
| full_name | roll_no |
+-----+-----+
| Aditi Gupta | 10A1001 |
| Vibeen Meir | 10A1002 |
| Aditi Pillai | 10A1003 |
| Sana Shett | 10A1004 |
+-----+-----+
```

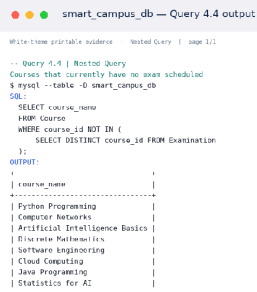
Output 4.3.1. Verified MySQL output for Query 4.3.

[PRINT THIS ASSET: [assets/query_outputs/q04_03_nested_query_output_01.png](#)]

Query 4.4 – Courses that currently have no exam scheduled

Category: Nested Query. Purpose: Courses that currently have no exam scheduled. The statement below is taken from the published SQL source.

```
01 SELECT course_name
02 FROM Course
03 WHERE course_id NOT IN (
04     SELECT DISTINCT course_id FROM Examination
05 );
```



smart_campus_db — Query 4.4 output

Write-time portable evidence | Nested Query | page 1/1

```
-- Query 4.4 | Nested Query
Courses that currently have no exam scheduled
$ mysql --table -D smart_campus_db
SQL:
SELECT course_name
FROM Course
WHERE course_id NOT IN (
    SELECT DISTINCT course_id FROM Examination
);
OUTPUT:
+-----+
| course_name |
+-----+
| Python Programming |
| Computer Networks |
| Artificial Intelligence Basics |
| Discrete Mathematics |
| Software Engineering |
| Cloud Computing |
| Java Programming |
| Statistics For AI |
+-----+
```

Output 4.4.1. Verified MySQL output for Query 4.4.

[PRINT THIS ASSET: [assets/query_outputs/q04_04_nested_query_output_01.png](#)]

PART E – SQL QUERY EVIDENCE

Query 4.5 – Student with the highest marks in each exam (correlated subquery)

Category: Nested Query. Purpose: Student with the highest marks in each exam (correlated subquery). The statement below is taken from the published SQL source.

```
01 SELECT e.exam_name, s.full_name, r.marks_obtained
02 FROM Result r
03 JOIN Examination e ON r.exam_id = e.exam_id
04 JOIN Student s ON r.student_id = s.student_id
05 WHERE r.marks_obtained = (
06     SELECT MAX(r2.marks_obtained) FROM Result r2 WHERE r2.exam_id = r.exam_id
07 );
08
09
10 -- *****
11 -- 5. UPDATE QUERIES (5)
12 -- *****
```

smart_campus_db — Query 4.5 output

Write these printable evidence | Nested Query | page 1/1

```
-- Query 4.5 | Nested Query
Student with the highest marks in each exam (correlated subquery)
$ mysql --table -D smart_campus_db
SQL
SELECT e.exam_name, s.full_name, r.marks_obtained
FROM Result r
JOIN Examination e ON r.exam_id = e.exam_id
JOIN Student s ON r.student_id = s.student_id
WHERE r.marks_obtained = (
    SELECT MAX(r2.marks_obtained) FROM Result r2 WHERE r2.exam_id = r.exam_id
);

*****
-- 5. UPDATE QUERIES (5)
-- *****

OUTPUT:
+-----+
| exam_name | full_name | marks_obtained |
+-----+
| Mid Semester Exam | Krishna Mahita | 80.40 |
| End Semester Exam | Krishna Mahita | 70.40 |
| Unit Test 1 | Neha Bhutt | 91.00 |
| End Semester Exam | Dev Sharma | 56.00 |
| Mid Semester Exam | Ananya P. Lal | 84.00 |
| Unit Test 1 | Zhaan Hair | 91.20 |
| Unit Test 2 | Reyansh Rai | 42.50 |
| Mid Semester Exam | Aditi Gupta | 41.00 |
| Unit Test 1 | Neha Bhutt | 92.00 |
| End Semester Exam | Ananya Pathotra | 70.00 |
| Unit Test 1 | Dev Sharma | 61.00 |
| Unit Test 2 | Ananya Pathotra | 79.00 |
| Unit Test 1 | Krishna Mahita | 49.00 |
| Unit Test 2 | Ananya Pathotra | 53.10 |
+-----+
```

Output 4.5.1. Verified MySQL output for Query 4.5.

[PRINT THIS ASSET: [assets/query_outputs/q04_05_nested_query_output_01.png](#)]

Query 5.1 – Update a student's cgpa after result declaration

Category: UPDATE. Purpose: Update a student's cgpa after result declaration. The statement below is taken from the published SQL source.

```
01 UPDATE Student SET cgpa = 9.10 WHERE roll_no = 'FDAI001';
```

smart_campus_db — Query 5.1 output

Write these printable evidence | UPDATE | page 1/1

```
-- Query 5.1 | UPDATE
Update a student's cgpa after result declaration
$ mysql --table -D smart_campus_db
SQL
UPDATE Student SET cgpa = 9.10 WHERE roll_no = 'FDAI001';

OUTPUT:
+-----+
| affected_rows |
+-----+
| 1 |
+-----+

+-----+
| roll_no | cgpa |
+-----+
| FDAI001 | 9.10 |
+-----+
```

Output 5.1.1. Verified MySQL output for Query 5.1.

[PRINT THIS ASSET: [assets/query_outputs/q05_01_update_output_01.png](#)]

Manual-writing note

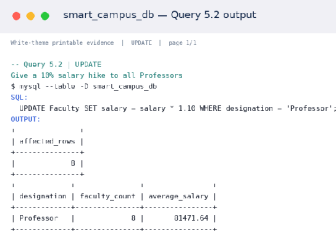
Write the query purpose and result interpretation. Print the marked output screenshot as evidence; the affected-row or schema result is already visible in the attached asset.

PART E – SQL QUERY EVIDENCE

Query 5.2 – Give a 10% salary hike to all Professors

Category: UPDATE. Purpose: Give a 10% salary hike to all Professors. The statement below is taken from the published SQL source.

```
01 UPDATE Faculty SET salary = salary * 1.10 WHERE designation = 'Professor';
```



```
smart_campus_db — Query 5.2 output
Write these printable evidence | UPDATE | page 1/1

-- Query 5.2 : UPDATE
Give a 10% salary hike to all Professors
$ mysql --table -D smart_campus_db
SQL
UPDATE Faculty SET salary = salary * 1.10 WHERE designation = 'Professor';
OUTPUT:
+-----+
| affected_rows |
+-----+
| 0 |
+-----+

+-----+
| designation | faculty_count | average_salary |
+-----+
| Professor | 0 | 31473.64 |
+-----+
```

Output 5.2.1. Verified MySQL output for Query 5.2.

[PRINT THIS ASSET: assets/query_outputs/q05_02_update_output_01.png]

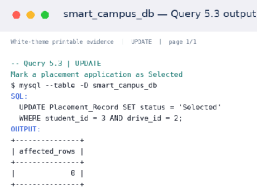
Manual-writing note

Write the query purpose and result interpretation. Print the marked output screenshot as evidence; the affected-row or schema result is already visible in the attached asset.

Query 5.3 – Mark a placement application as Selected

Category: UPDATE. Purpose: Mark a placement application as Selected. The statement below is taken from the published SQL source.

```
01 UPDATE Placement_Record SET status = 'Selected'
02 WHERE student_id = 3 AND drive_id = 2;
```



```
smart_campus_db — Query 5.3 output
Write these printable evidence | UPDATE | page 1/1

-- Query 5.3 : UPDATE
Mark a placement application as Selected
$ mysql --table -D smart_campus_db
SQL
UPDATE Placement_Record SET status = 'Selected'
WHERE student_id = 3 AND drive_id = 2;
OUTPUT:
+-----+
| affected_rows |
+-----+
| 0 |
+-----+
```

Output 5.3.1. Verified MySQL output for Query 5.3.

[PRINT THIS ASSET: assets/query_outputs/q05_03_update_output_01.png]

Manual-writing note

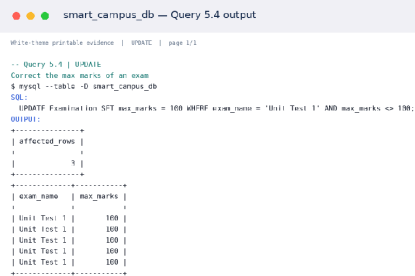
Write the query purpose and result interpretation. Print the marked output screenshot as evidence; the affected-row or schema result is already visible in the attached asset.

PART E – SQL QUERY EVIDENCE

Query 5.4 – Correct the max marks of an exam

Category: UPDATE. Purpose: Correct the max marks of an exam. The statement below is taken from the published SQL source.

```
01 UPDATE Examination SET max_marks = 100 WHERE exam_name = 'Unit Test 1' AND max_marks <> 100;
```



```
smart Campus_db — Query 5.4 output
Write these printable evidence | UPDATE | page 1/1
-- Query 5.4 | UPDATE
Correct the max marks of an exam
$ mysql --table -D smart_campus_db
SQL
UPDATE Examination SET max_marks = 100 WHERE exam_name = 'Unit Test 1' AND max_marks <> 100;
OUTPUT:
+-----+
| affected_rows |
+-----+
| 1 |
+-----+
+-----+
| exam_name | max_marks |
+-----+
| Unit Test 1 | 100 |
| Unit Test 1 | 100 |
| Unit Test 1 | 100 |
| Unit Test 1 | 100 |
| Unit Test 1 | 100 |
+-----+
```

Output 5.4.1. Verified MySQL output for Query 5.4.

[PRINT THIS ASSET: [assets/query_outputs/q05_04_update_output_01.png](#)]

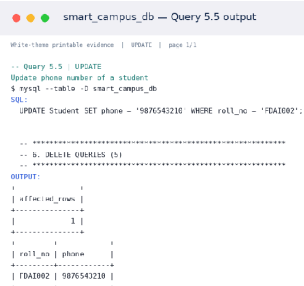
Manual-writing note

Write the query purpose and result interpretation. Print the marked output screenshot as evidence; the affected-row or schema result is already visible in the attached asset.

Query 5.5 – Update phone number of a student

Category: UPDATE. Purpose: Update phone number of a student. The statement below is taken from the published SQL source.

```
01 UPDATE Student SET phone = '9876543210' WHERE roll_no = 'FDAI002';
02
03
04 -- *****
05 -- 6. DELETE QUERIES (5)
06 -- *****
```



```
smart Campus_db — Query 5.5 output
Write these printable evidence | UPDATE | page 1/1
-- Query 5.5 | UPDATE
Update phone number of a student
$ mysql --table -D smart_campus_db
SQL
UPDATE Student SET phone = '9876543210' WHERE roll_no = 'FDAI002';
*****
-- b. Health QUERIES (5)
*****
OUTPUT:
+-----+
| affected_rows |
+-----+
| 1 |
+-----+
+-----+
| roll_no | phone |
+-----+
| FDAI002 | 9876543210 |
+-----+
```

Output 5.5.1. Verified MySQL output for Query 5.5.

[PRINT THIS ASSET: [assets/query_outputs/q05_05_update_output_01.png](#)]

Manual-writing note

Write the query purpose and result interpretation. Print the marked output screenshot as evidence; the affected-row or schema result is already visible in the attached asset.

PART E – SQL QUERY EVIDENCE

Query 6.1 – Remove a single wrong attendance entry

Category: DELETE. Purpose: Remove a single wrong attendance entry. The statement below is taken from the published SQL source.

```
01 DELETE FROM Attendance WHERE attendance_id = 30;
```

smart_campus_db — Query 6.1 output

Write three printable evidence | DELETE | page 1/1

```
-- Query 6.1 : DELETE
Remove a single wrong attendance entry
$ mysql --local -D smart_campus_db
SQL>
DELETE FROM Attendance WHERE attendance_id = 30;
OUTPUT:
+-----+
| affected_rows |
+-----+
|             1 |
+-----+
+-----+
| remaining_attendance |
+-----+
|                   29 |
+-----+
```

Output 6.1.1. Verified MySQL output for Query 6.1.

[PRINT THIS ASSET: [assets/query_outputs/q06_01_delete_output_01.png](#)]

Manual-writing note

Write the query purpose and result interpretation. Print the marked output screenshot as evidence; the affected-row or schema result is already visible in the attached asset.

Query 6.2 – Remove placement applications that were rejected before 2026

Category: DELETE. Purpose: Remove placement applications that were rejected before 2026. The statement below is taken from the published SQL source.

```
01 DELETE FROM Placement_Record WHERE status = 'Rejected' AND placement_id = 20;
```

smart_campus_db — Query 6.2 output

Write three printable evidence | DELETE | page 1/1

```
-- Query 6.2 : DELETE
Remove placement applications that were rejected before 2026
$ mysql --local -D smart_campus_db
SQL>
DELETE FROM Placement_Record WHERE status = 'Rejected' AND placement_id = 20;
OUTPUT:
+-----+
| affected_rows |
+-----+
|             1 |
+-----+
+-----+
| remaining_placement_records |
+-----+
|                   20 |
+-----+
```

Output 6.2.1. Verified MySQL output for Query 6.2.

[PRINT THIS ASSET: [assets/query_outputs/q06_02_delete_output_01.png](#)]

Manual-writing note

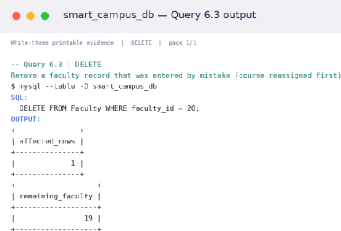
Write the query purpose and result interpretation. Print the marked output screenshot as evidence; the affected-row or schema result is already visible in the attached asset.

PART E – SQL QUERY EVIDENCE

Query 6.3 — Remove a faculty record that was entered by mistake (course reassigned first)

Category: DELETE. Purpose: Remove a faculty record that was entered by mistake (course reassigned first). The statement below is taken from the published SQL source.

```
01 DELETE FROM Faculty WHERE faculty_id = 20;
```



```
smart_campus_db — Query 6.3 output
Write these portable evidence | DELETE | page 1/1

-- Query 6.3 | DELETE
Remove a faculty record that was entered by mistake (course reassigned first)
$ mysql --local -D smart_campus_db
SQL>
DELETE FROM Faculty WHERE faculty_id = 20;
OUTPUT:
+-----+
| affected_rows |
+-----+
| 1 |
+-----+
+-----+
| remaining_faculty |
+-----+
| 19 |
+-----+
```

Output 6.3.1. Verified MySQL output for Query 6.3.

[PRINT THIS ASSET: [assets/query_outputs/q06_03_delete_output_01.png](#)]

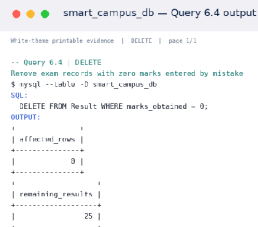
Manual-writing note

Write the query purpose and result interpretation. Print the marked output screenshot as evidence; the affected-row or schema result is already visible in the attached asset.

Query 6.4 — Remove exam records with zero marks entered by mistake

Category: DELETE. Purpose: Remove exam records with zero marks entered by mistake. The statement below is taken from the published SQL source.

```
01 DELETE FROM Result WHERE marks_obtained = 0;
```



```
smart_campus_db — Query 6.4 output
Write these portable evidence | DELETE | page 1/1

-- Query 6.4 | DELETE
Remove exam records with zero marks entered by mistake
$ mysql --local -D smart_campus_db
SQL>
DELETE FROM Result WHERE marks_obtained = 0;
OUTPUT:
+-----+
| affected_rows |
+-----+
| 0 |
+-----+
+-----+
| remaining_results |
+-----+
| 25 |
+-----+
```

Output 6.4.1. Verified MySQL output for Query 6.4.

[PRINT THIS ASSET: [assets/query_outputs/q06_04_delete_output_01.png](#)]

Manual-writing note

Write the query purpose and result interpretation. Print the marked output screenshot as evidence; the affected-row or schema result is already visible in the attached asset.

PART E – SQL QUERY EVIDENCE

Query 6.5 — Remove a placement drive that got cancelled

Category: DELETE. Purpose: Remove a placement drive that got cancelled. The statement below is taken from the published SQL source.

```
01 DELETE FROM Placement_Drive WHERE drive_id = 15;
02
03
04 -- *****
05 -- 7. ALTER TABLE QUERIES (5)
06 -- *****
```

smart_campus_db — Query 6.5 output

```
Write these printable evidence | DELETE | page 1/1

-- Query 6.5 | DELETE
Remove a placement drive that got cancelled
$ mysql --table -D smart_campus_db
SQL
DELETE FROM Placement_Drive WHERE drive_id = 15;

-----
-- 7. ALTER TABLE QUERIES (5)
--
OUTPUT:
+-----+
| affected_rows |
+-----+
| 1 |
+-----+

+-----+
| remaining_drives |
+-----+
| 14 |
+-----+
```

Output 6.5.1. Verified MySQL output for Query 6.5.

[PRINT THIS ASSET: [assets/query_outputs/q06_05_delete_output_01.png](#)]

Manual-writing note

Write the query purpose and result interpretation. Print the marked output screenshot as evidence; the affected-row or schema result is already visible in the attached asset.

Query 7.1 — Add a new column to store student's blood group

Category: ALTER TABLE. Purpose: Add a new column to store student's blood group. The statement below is taken from the published SQL source.

```
01 ALTER TABLE Student ADD COLUMN blood_group VARCHAR(5);
```

smart_campus_db — Query 7.1 output

```
Write these printable evidence | ALTER TABLE | page 1/1

-- Query 7.1 | ALTER TABLE
Add a new column to store student's blood group
$ mysql --table -D smart_campus_db
SQL
ALTER TABLE Student ADD COLUMN blood_group VARCHAR(5);

OUTPUT:
+-----+
| affected_rows |
+-----+
| 0 |
+-----+

+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+
| blood_group | varchar(5) | YES | | NULL | |
+-----+
```

Output 7.1.1. Verified MySQL output for Query 7.1.

[PRINT THIS ASSET: [assets/query_outputs/q07_01_alter_table_output_01.png](#)]

Manual-writing note

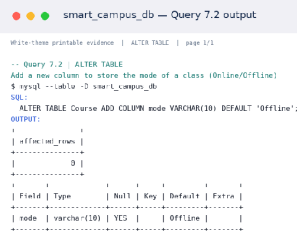
Write the query purpose and result interpretation. Print the marked output screenshot as evidence; the affected-row or schema result is already visible in the attached asset.

PART E – SQL QUERY EVIDENCE

Query 7.2 – Add a new column to store the mode of a class (Online/Offline)

Category: ALTER TABLE. Purpose: Add a new column to store the mode of a class (Online/Offline). The statement below is taken from the published SQL source.

```
01 ALTER TABLE Course ADD COLUMN mode VARCHAR(10) DEFAULT 'Offline';
```



```
smart_campus_db — Query 7.2 output
Write these portable evidence | ALTER TABLE | page 1/1

-- Query 7.2 | ALTER TABLE
Add a new column to store the mode of a class (Online/Offline)
$ mysql --table -D smart_campus_db
SQL:
ALTER TABLE Course ADD COLUMN mode VARCHAR(10) DEFAULT 'Offline';
OUTPUT:
+-----+
| affected_rows |
+-----+
| 0 |
+-----+

+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+
| mode | varchar(10) | YES | | Offline | |
+-----+
```

Output 7.2.1. Verified MySQL output for Query 7.2.

[PRINT THIS ASSET: [assets/query_outputs/q07_02_alter_table_output_01.png](#)]

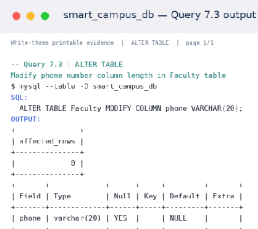
Manual-writing note

Write the query purpose and result interpretation. Print the marked output screenshot as evidence; the affected-row or schema result is already visible in the attached asset.

Query 7.3 – Modify phone number column length in Faculty table

Category: ALTER TABLE. Purpose: Modify phone number column length in Faculty table. The statement below is taken from the published SQL source.

```
01 ALTER TABLE Faculty MODIFY COLUMN phone VARCHAR(20);
```



```
smart_campus_db — Query 7.3 output
Write these portable evidence | ALTER TABLE | page 1/1

-- Query 7.3 | ALTER TABLE
Modify phone number column length in Faculty table
$ mysql --table -D smart_campus_db
SQL:
ALTER TABLE Faculty MODIFY COLUMN phone VARCHAR(20);
OUTPUT:
+-----+
| affected_rows |
+-----+
| 0 |
+-----+

+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+
| phone | varchar(20) | YES | | NULL | |
+-----+
```

Output 7.3.1. Verified MySQL output for Query 7.3.

[PRINT THIS ASSET: [assets/query_outputs/q07_03_alter_table_output_01.png](#)]

Manual-writing note

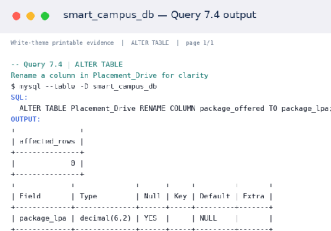
Write the query purpose and result interpretation. Print the marked output screenshot as evidence; the affected-row or schema result is already visible in the attached asset.

PART E – SQL QUERY EVIDENCE

Query 7.4 – Rename a column in Placement_Drive for clarity

Category: ALTER TABLE. Purpose: Rename a column in Placement_Drive for clarity. The statement below is taken from the published SQL source.

```
01 ALTER TABLE Placement_Drive RENAME COLUMN package_offered TO package_lpa;
```



```
smart_campus_db — Query 7.4 output
Write these printable evidence | ALTER TABLE | page 1/1

-- Query 7.4 : ALTER TABLE
Rename a column in Placement_Drive for clarity
$ mysql --label -D smart_campus_db
SQL:
ALTER TABLE Placement_Drive RENAME COLUMN package_offered TO package_lpa;
OUTPUT:
+-----+
| affected_rows |
+-----+
| 0 |
+-----+

+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+
| package_lpa | decimal(6,2) | YES | | NULL | |
+-----+
```

Output 7.4.1. Verified MySQL output for Query 7.4.

[PRINT THIS ASSET: [assets/query_outputs/q07_04_alter_table_output_01.png](#)]

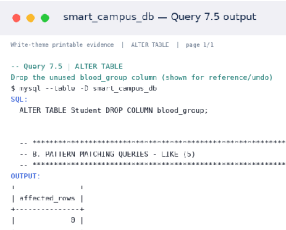
Manual-writing note

Write the query purpose and result interpretation. Print the marked output screenshot as evidence; the affected-row or schema result is already visible in the attached asset.

Query 7.5 – Drop the unused blood_group column (shown for reference/undo)

Category: ALTER TABLE. Purpose: Drop the unused blood_group column (shown for reference/undo). The statement below is taken from the published SQL source.

```
01 ALTER TABLE Student DROP COLUMN blood_group;
02
03
04 -- *****
05 -- 8. PATTERN MATCHING QUERIES - LIKE (5)
06 -- *****
```



```
smart_campus_db — Query 7.5 output
Write these printable evidence | ALTER TABLE | page 1/1

-- Query 7.5 : ALTER TABLE
Drop the unused blood_group column (shown for reference/undo)
$ mysql --label -D smart_campus_db
SQL:
ALTER TABLE Student DROP COLUMN blood_group;
OUTPUT:
+-----+
| affected_rows |
+-----+
| 0 |
+-----+
```

Output 7.5.1. Verified MySQL output for Query 7.5.

[PRINT THIS ASSET: [assets/query_outputs/q07_05_alter_table_output_01.png](#)]

Manual-writing note

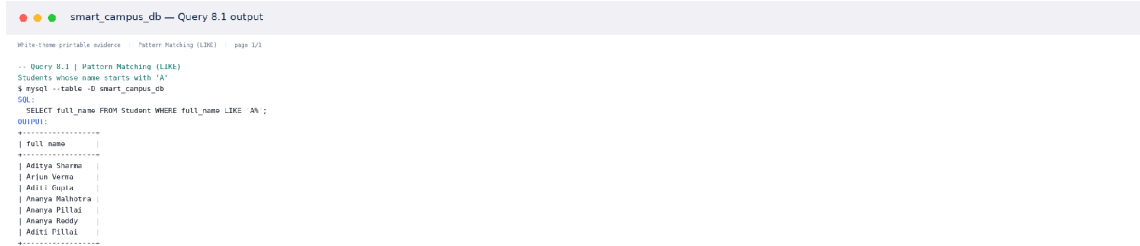
Write the query purpose and result interpretation. Print the marked output screenshot as evidence; the affected-row or schema result is already visible in the attached asset.

PART E – SQL QUERY EVIDENCE

Query 8.1 – Students whose name starts with 'A'

Category: Pattern Matching (LIKE). Purpose: Students whose name starts with 'A'. The statement below is taken from the published SQL source.

```
01 SELECT full_name FROM Student WHERE full_name LIKE 'A%';
```



```
smart_campus_db — Query 8.1 output
Write-time portable evidence | Pattern Matching (LIKE) | page 3/3

-- Query 8.1 | Pattern Matching (LIKE)
Students whose name starts with 'A'
$ mysql --table -D smart_campus_db
SQL:
SELECT full_name FROM Student WHERE full_name LIKE 'A%';
OUTPUT:
+-----+
| full_name |
+-----+
| Aditya Sharma |
| Arjun Verma |
| Aditi Chouda |
| Ananya Mahabatra |
| Ananya Pillai |
| Ananya Rastogi |
| Aditi Pillai |
+-----+
```

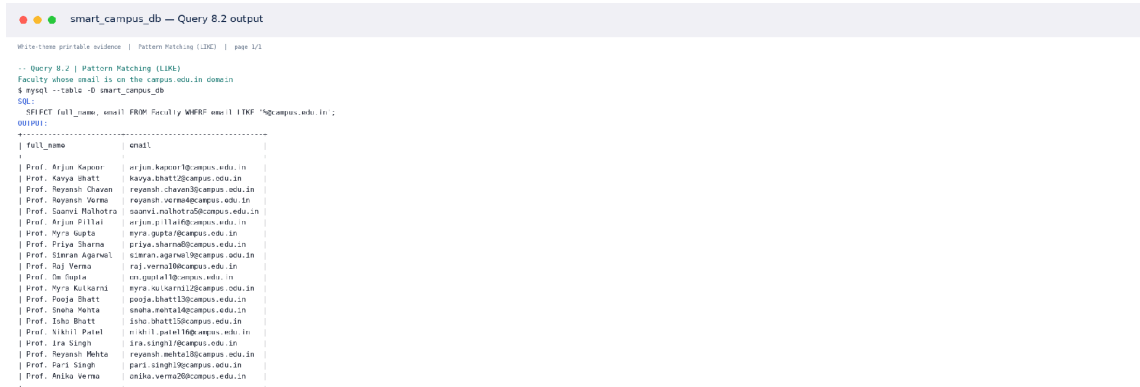
Output 8.1.1. Verified MySQL output for Query 8.1.

[PRINT THIS ASSET: [assets/query_outputs/q08_01_like_output_01.png](#)]

Query 8.2 – Faculty whose email is on the campus.edu.in domain

Category: Pattern Matching (LIKE). Purpose: Faculty whose email is on the campus.edu.in domain. The statement below is taken from the published SQL source.

```
01 SELECT full_name, email FROM Faculty WHERE email LIKE '%@campus.edu.in';
```



```
smart_campus_db — Query 8.2 output
Write-time portable evidence | Pattern Matching (LIKE) | page 3/3

-- Query 8.2 | Pattern Matching (LIKE)
Faculty whose email is on the campus.edu.in domain
$ mysql --table -D smart_campus_db
SQL:
SELECT full_name, email FROM Faculty WHERE email LIKE '%@campus.edu.in';
OUTPUT:
+-----+-----+
| full_name | email |
+-----+-----+
| Prof. Arjun Kumar | arjun.kumar10@campus.edu.in |
| Prof. Kanya Bhatt | kanya.bhatt1@campus.edu.in |
| Prof. Rayansh Chavan | rayansh.chavan1@campus.edu.in |
| Prof. Rayansh Verma | rayansh.verma1@campus.edu.in |
| Prof. Saanvi Mahabatra | saanvi.mahabatra1@campus.edu.in |
| Prof. Arjun Pillai | arjun.pillai1@campus.edu.in |
| Prof. Myra Gupta | myra.gupta1@campus.edu.in |
| Prof. Priya Sharma | priya.sharma1@campus.edu.in |
| Prof. Simeran Agarwal | simran.agarwal1@campus.edu.in |
| Prof. Raj Verma | raj.verma1@campus.edu.in |
| Prof. De Duple | de.duple1@campus.edu.in |
| Prof. Myra Kulkarni | myra.kulkarni1@campus.edu.in |
| Prof. Pooja Bhatt | pooja.bhatt1@campus.edu.in |
| Prof. Sneha Nanta | sneha.nanta1@campus.edu.in |
| Prof. Isha Bhatt | isha.bhatt1@campus.edu.in |
| Prof. Nikhil Patel | nikhil.patel1@campus.edu.in |
| Prof. Isha Singh | isha.singh1@campus.edu.in |
| Prof. Rayansh Mehta | rayansh.mehta1@campus.edu.in |
| Prof. Pari Singh | pari.singh1@campus.edu.in |
| Prof. Anika Verma | anika.verma1@campus.edu.in |
+-----+-----+
```

Output 8.2.1. Verified MySQL output for Query 8.2.

[PRINT THIS ASSET: [assets/query_outputs/q08_02_like_output_01.png](#)]

PART E – SQL QUERY EVIDENCE

Query 8.3 – Courses with 'Data' anywhere in the course name

Category: Pattern Matching (LIKE). Purpose: Courses with 'Data' anywhere in the course name. The statement below is taken from the published SQL source.

```
01 SELECT course_name FROM Course WHERE course_name LIKE '%Data%';
```

smart_campus_db — Query 8.3 output

Write-time portable evidence | Pattern Matching (LIKE) | page 3/3

```
-- Query 8.3 | Pattern Matching (LIKE)
Courses with 'Data' anywhere in the course name
$ mysql --table -d smart_campus_db
SQL:
SELECT course_name FROM Course WHERE course_name LIKE '%Data%';
OUTPUT:
+-----+
| course_name |
+-----+
| Database Management Systems |
| Data Structures |
| Data Analytics |
+-----+
```

Output 8.3.1. Verified MySQL output for Query 8.3.

[PRINT THIS ASSET: [assets/query_outputs/q08_03_like_output_01.png](#)]

Query 8.4 – Companies whose name ends with 'a'

Category: Pattern Matching (LIKE). Purpose: Companies whose name ends with 'a'. The statement below is taken from the published SQL source.

```
01 SELECT company_name FROM Placement_Drive WHERE company_name LIKE '%a';
```

smart_campus_db — Query 8.4 output

Write-time portable evidence | Pattern Matching (LIKE) | page 3/3

```
-- Query 8.4 | Pattern Matching (LIKE)
Companies whose name ends with 'a'
$ mysql --table -d smart_campus_db
SQL:
SELECT company_name FROM Placement_Drive WHERE company_name LIKE '%a';
OUTPUT:
+-----+
| company_name |
+-----+
| IBM India |
| Tech Mahindra |
+-----+
```

Output 8.4.1. Verified MySQL output for Query 8.4.

[PRINT THIS ASSET: [assets/query_outputs/q08_04_like_output_01.png](#)]

Query 8.5 – Students whose roll number matches the pattern FDAIO_1 (2nd last digit is 1)

Category: Pattern Matching (LIKE). Purpose: Students whose roll number matches the pattern FDAIO_1 (2nd last digit is 1). The statement below is taken from the published SQL source.

```
01 SELECT roll_no, full_name FROM Student WHERE roll_no LIKE 'FDAIO_1';
```

smart_campus_db — Query 8.5 output

Write-time portable evidence | Pattern Matching (LIKE) | page 3/3

```
-- Query 8.5 | Pattern Matching (LIKE)
Students whose roll number matches the pattern FDAIO_1 (2nd last digit is 1)
$ mysql --table -d smart_campus_db
SQL:
SELECT roll_no, full_name FROM Student WHERE roll_no LIKE 'FDAIO_1';
OUTPUT:
+-----+
| roll_no | full_name |
+-----+
| FDAIO01 | Krishna Sharma |
| FDAIO11 | Neha Bhatt |
+-----+
```

Output 8.5.1. Verified MySQL output for Query 8.5.

[PRINT THIS ASSET: [assets/query_outputs/q08_05_like_output_01.png](#)]

PART F – REPORT

F1. Challenges Faced

Challenge	Short solution
Referential integrity	Created parent tables first and used foreign keys.
Duplicate data	Used UNIQUE constraints and 3NF separation.
Child-record handling	Applied ON DELETE CASCADE or SET NULL.
Many query categories	Organized the source into eight clearly labelled groups.
Large evidence volume	Placed every output screenshot in a systematic assets folder.

F2. Learning Outcomes

- Understood requirement analysis, entity identification, relationships, and relational schema design.
- Applied primary keys, foreign keys, unique values, mandatory fields, checks, defaults, and cascading actions.
- Practised retrieval, aggregate, JOIN, nested, UPDATE, DELETE, ALTER TABLE, and LIKE operations.
- Learned to verify SQL output and document query results with reproducible evidence.

F3. Conclusion

The Smart Campus Management System provides a structured relational solution for academic and placement operations. The project satisfies the teacher's Parts A–F requirements and combines handwritten-friendly explanations with printable SQL and output evidence.

PRESENTATION REFERENCE – 5-MINUTE OUTLINE

The official brief asks for a short presentation reference. The following outline can be used to prepare five concise slides without adding unnecessary written work to the report.

Time	Slide / topic	Key content
0:00–0:45	Problem and objective	Why a Smart Campus database is required.
0:45–1:30	Selected modules	Student, Faculty, Attendance, Examination/Result, Placement.
1:30–2:15	Design	ER diagram, keys, relationships, and 3NF.
2:15–3:30	Implementation	Nine tables, constraints, seed records, and query categories.
3:30–4:30	Query evidence	Retrieval, aggregate, JOIN, nested, DML, ALTER, and LIKE examples.
4:30–5:00	Conclusion	Challenges, learning outcomes, and future scope.

Presentation reminder

Use the report's ER diagram, workflow, and output screenshots as visual references. The presentation is separate from the handwritten report and SQL printout.

MANUAL WRITING AND PRINTING GUIDE

Use this page as a final preparation guide. The explanatory paragraphs and tables are intended for manual writing; the marked source-code and output screenshots are intended for printing and attachment.

Report part	Write manually	Print and attach
Part A	Objectives, users, functional and non-functional requirements	None required unless teacher asks
Part B	Selected modules, schema explanation, key mapping, 3NF summary	ER and workflow diagrams
Part C	Database setup explanation and constraint meanings	SQL source/code screenshots
Part D	Insert order and data-entry strategy	Optional record-count/output evidence
Part E	Query title, purpose, SQL statement, and short interpretation	The matching output screenshot below every query
Part F	Challenges, learning outcomes, conclusion, references	Final report and selected supporting assets

Asset naming rule

Every screenshot filename begins with the query number, for example q01_01_retrieval_output_01.png. This makes it easy to print the exact file shown below each query.

PRINTABLE ASSET INDEX

The following files are stored systematically inside the assets folder. Print the required files and attach them; do not copy the image filenames into the handwritten body.

Source-Code Screenshots

File	Print marker
code_01_lines_001_048.png	[PRINT THIS ASSET: assets/code_screenshots/code_01_lines_001_048.png]
code_02_lines_049_096.png	[PRINT THIS ASSET: assets/code_screenshots/code_02_lines_049_096.png]
code_03_lines_097_144.png	[PRINT THIS ASSET: assets/code_screenshots/code_03_lines_097_144.png]
code_04_lines_145_192.png	[PRINT THIS ASSET: assets/code_screenshots/code_04_lines_145_192.png]
code_05_lines_193_240.png	[PRINT THIS ASSET: assets/code_screenshots/code_05_lines_193_240.png]
code_06_lines_241_288.png	[PRINT THIS ASSET: assets/code_screenshots/code_06_lines_241_288.png]
code_07_lines_289_336.png	[PRINT THIS ASSET: assets/code_screenshots/code_07_lines_289_336.png]
code_08_lines_337_384.png	[PRINT THIS ASSET: assets/code_screenshots/code_08_lines_337_384.png]
code_09_lines_385_432.png	[PRINT THIS ASSET: assets/code_screenshots/code_09_lines_385_432.png]
code_10_lines_433_480.png	[PRINT THIS ASSET: assets/code_screenshots/code_10_lines_433_480.png]
code_11_lines_481_528.png	[PRINT THIS ASSET: assets/code_screenshots/code_11_lines_481_528.png]
code_12_lines_529_566.png	[PRINT THIS ASSET: assets/code_screenshots/code_12_lines_529_566.png]

Query-Output Screenshot Summary

Query	Asset location
Query 1.1	1 output asset(s) — assets/query_outputs/
Query 1.2	1 output asset(s) — assets/query_outputs/
Query 1.3	1 output asset(s) — assets/query_outputs/
Query 1.4	1 output asset(s) — assets/query_outputs/
Query 1.5	1 output asset(s) — assets/query_outputs/
Query 1.6	1 output asset(s) — assets/query_outputs/
Query 1.7	1 output asset(s) — assets/query_outputs/
Query 1.8	1 output asset(s) — assets/query_outputs/
Query 1.9	1 output asset(s) — assets/query_outputs/
Query 1.10	1 output asset(s) — assets/query_outputs/
Query 2.1	1 output asset(s) — assets/query_outputs/
Query 2.2	1 output asset(s) — assets/query_outputs/

Query	Asset location
Query 2.3	1 output asset(s) — assets/query_outputs/
Query 2.4	1 output asset(s) — assets/query_outputs/
Query 2.5	1 output asset(s) — assets/query_outputs/
Query 3.1	2 output asset(s) — assets/query_outputs/
Query 3.2	1 output asset(s) — assets/query_outputs/
Query 3.3	1 output asset(s) — assets/query_outputs/
Query 3.4	1 output asset(s) — assets/query_outputs/
Query 3.5	1 output asset(s) — assets/query_outputs/
Query 4.1	1 output asset(s) — assets/query_outputs/
Query 4.2	1 output asset(s) — assets/query_outputs/
Query 4.3	1 output asset(s) — assets/query_outputs/
Query 4.4	1 output asset(s) — assets/query_outputs/
Query 4.5	1 output asset(s) — assets/query_outputs/
Query 5.1	1 output asset(s) — assets/query_outputs/
Query 5.2	1 output asset(s) — assets/query_outputs/
Query 5.3	1 output asset(s) — assets/query_outputs/
Query 5.4	1 output asset(s) — assets/query_outputs/
Query 5.5	1 output asset(s) — assets/query_outputs/
Query 6.1	1 output asset(s) — assets/query_outputs/
Query 6.2	1 output asset(s) — assets/query_outputs/
Query 6.3	1 output asset(s) — assets/query_outputs/
Query 6.4	1 output asset(s) — assets/query_outputs/
Query 6.5	1 output asset(s) — assets/query_outputs/
Query 7.1	1 output asset(s) — assets/query_outputs/
Query 7.2	1 output asset(s) — assets/query_outputs/
Query 7.3	1 output asset(s) — assets/query_outputs/
Query 7.4	1 output asset(s) — assets/query_outputs/
Query 7.5	1 output asset(s) — assets/query_outputs/
Query 8.1	1 output asset(s) — assets/query_outputs/
Query 8.2	1 output asset(s) — assets/query_outputs/
Query 8.3	1 output asset(s) — assets/query_outputs/
Query 8.4	1 output asset(s) — assets/query_outputs/
Query 8.5	1 output asset(s) — assets/query_outputs/

Final submission checklist

Attach the complete SQL file, the marked code/output screenshots, and this DOCX reference report. The README.md in the project root mirrors this documentation for GitHub users.

REFERENCES

- [1] Teacher-provided assignment brief: FYAIDBMSASSIGNMENT.pdf.
- [2] Published project source: <https://github.com/codeXparadise/Assignments/tree/main/DBMS/UNIT%201>.
- [3] MySQL 8.0 Reference Manual: <https://dev.mysql.com/doc/refman/8.0/en/>.
- [4] Third Normal Form reference: https://en.wikipedia.org/wiki/Third_normal_form.

Prepared as a student reference document for the Smart Campus Management System DBMS assignment.